

Wazuh SOC Monitoring & Custom Detection Rules

Summary

This document records the Wazuh-based SOC lab work completed as part of a hands-on cybersecurity practice environment. The lab focuses on endpoint monitoring, Apache/DVWA log collection, File Integrity Monitoring (FIM), custom detection rules, MITRE ATT&CK mapping, and automated firewall containment.

Note: IP addresses have been replaced with placeholders to protect local lab network details. The lab was conducted in a controlled virtual environment.

1. Project Overview

The objective was to build and test a small SOC monitoring environment using Wazuh, with an Ubuntu server hosting DVWA/Apache monitored by a Wazuh agent and a separate Wazuh manager, indexer, and dashboard. Kali Linux was used as the attacker system.

2. Lab Architecture

Component	Role	IP / Identifier
Kali Linux	Simulated attacker	ATTACKER IP
Ubuntu DVWA host	Apache + DVWA + Wazuh Agent	VICTIM IP
Wazuh server	Manager + Indexer + Dashboard	SIEM IP
Wazuh Agent	Monitors DVWA Ubuntu host	Agent name: DVWA-Ubuntu

3. Wazuh Agent Configuration

The DVWA Ubuntu system was enrolled as a Wazuh agent and configured to collect Apache web-server logs in addition to endpoint telemetry.

Important configuration path: `/var/ossec/etc/ossec.conf`

Apache access and error logs were configured as monitored local files. This allowed web requests generated during DVWA testing to reach the Wazuh analysis engine.

4. Php Upload Detection – Custom Rule 100100

Purpose: Detect the creation of PHP files in the DVWA upload directory using Wazuh File Integrity Monitoring. This was designed to identify a potentially dangerous web-shell or uploaded PHP payload.
Rule ID: 100100 | Severity: Level 12 | Parent Rule: 554

```
<group name="syscheck,syscheck_entry_added,dvwa_upload,">
<rule id="100100" level="12">
<if_sid>554</if_sid>
<field name="file" type="pcr2">/var/www/html/DVWA/hackable/uploads/.*\.php$</field>
<description>PHP file uploaded to DVWA uploads folder: $(file)</description>
</rule>
</group>
```

When a newly detected file matches the DVWA upload directory and has a .php extension, rule 100100 will be triggered.

Export Formatted Reset view 641 available fields Columns Density 1 fields sorted Full screen				
timestamp	agent.name	rule.description	rule.level	rule.id
Aug 15, 2026 @ 21:30:45.0...	DVWA-Ubuntu	PHP file uploaded to DVWA uploads folder: /var/www/html/DVWA/hackable/uploads/wazuh.php	12	100100

5. PHP Web Access Detection - Custom Rule 100101

Purpose: Detect HTTP access to PHP files located in the DVWA upload directory. This complements the FIM rule: Rule 100100 identifies the uploaded PHP file, while Rule 100101 identifies subsequent web access to that file.

Rule ID: 100101 | Severity: Level 12 | Parent rule: 31108

```
<group name="web,accesslog,dvwa,">
<rule id="100101" level="12">
<if_sid>31108</if_sid>
<url type="pcre2">(?!)^/DVWA/hackable/uploads/.*\php$</url>
<description>PHP file accessed in DVWA uploads directory from $(srcip)</description>
</rule>
</group>
```

The rule matches the Apache web-access event and checks whether the requested URL is a PHP file under /DVWA/hackable/uploads/. The source IP is included in the alert description for investigation and response.

rule.id	100101
rule.level	12
rule.mail	true
rule.mitre.id	T1505.003
rule.mitre.tactic	Persistence
rule.mitre.technique	Web Shell

Field	Value
Source IP	ATTACKER IP
Protocol / method	GET
HTTP status	200
Requested URL	/DVWA/hackable/uploads/wazuh.php
Log source	/var/log/apache2/access.log
MITRE mapping	T1505.003 – Web Shell

6. MITRE ATT&CK Mapping

After validating the PHP upload and web-access detections, the suspicious activity was mapped to the relevant MITRE ATT&CK technique.

T1505.003 - Server Software Component: Web Shell

The mapping was based on the scenario in which a PHP file is placed in a web-accessible directory and subsequently accessed through the web server.

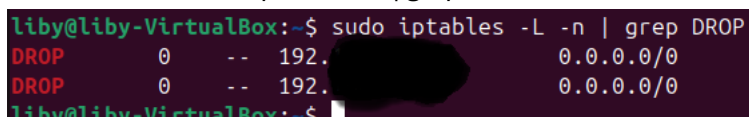
7. Active Response

Add the <active-response> block inside the <ossec_config> tag in the wazuh manager configuration file located at /var/ossec/etc/ossec.conf.

```
<active-response>
  <disabled>no</disabled>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100101</rules_id>
  <timeout>180</timeout>
</active-response>
```

Active response to block IP when accessing the php file. Wazuh includes firewall-drop as a default Active Response command on Linux. It can block the source IP associated with an alert. Timeout used for blocking IP for given time and unblock it after.

To confirm IP auto-block: `sudo iptables -L | grep DROP`



```
liby@liby-VirtualBox:~$ sudo iptables -L -n | grep DROP
DROP      0      --  192.168.1.101          0.0.0.0/0
DROP      0      --  192.168.1.101          0.0.0.0/0
liby@liby-VirtualBox:~$
```

Wazuh detected access to the uploaded PHP file through custom rule 100101, extracted the attacker source IP, and triggered the firewall-drop Active Response. The attacker IP was added to both the INPUT and FORWARD iptables chains and was automatically removed after the configured timeout.

8. SOC Skills Learned

- Wazuh manager/agent deployment and configuration
- Endpoint and Apache log monitoring
- File Integrity Monitoring (FIM)
- Custom Wazuh rule development using XML
- Regex based event matching
- Alert severity and rule chaining
- Web-server security monitoring
- MITRE ATT&CK technique mapping
- Automated incident containment with Wazuh Active Response
- Firewall-based source-IP blocking
- Testing and validation of detection/response logic